

VEDA SMART PARKING · RASPBERRY PI SERVER

Pi Server 코드 이해 가이드

폴더 · 파일 · 핵심 함수 · 런타임 흐름 종합 문서

대상: Hanwha Vision PNO-A9081R 기반 C++ 스마트 주차 서버

기술: C/C++ · MQTT · RTSP · OpenCV · Gemini HTTPS · SQLite · HTTP/HTTPS

기준일: 2026-07-22 · C++20 통합 서버 기준

생성: `cmake --build build --target docs`

문서 읽는 방법

1. 프로그램 전체 구조

2. 서버 시작과 종료 흐름

3. 카메라 이벤트·Snapshot 흐름

4. BestShot·OCR·EV 판별 흐름

5. 센서·주차 상태 흐름

6. 장기 점유 타이머 흐름

7. DB·HTTP/Qt 흐름

8. 폴더별 역할

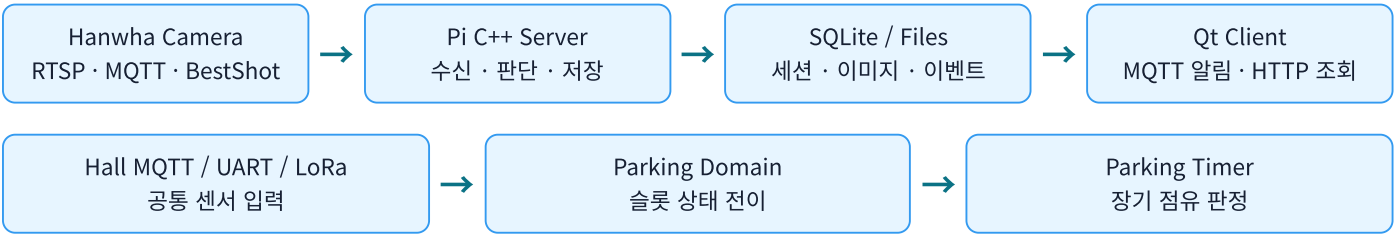
9. 파일별 역할

10. 핵심 함수 참조

11. 테스트와 디버깅

이 PDF는 구조와 책임을 학습하기 위한 안내서다. 세부 매개변수와 private 구현은 각 헤더의 Doxygen 형식 주석과 실제 소스 파일에서 확인한다.

1. 프로그램 전체 구조



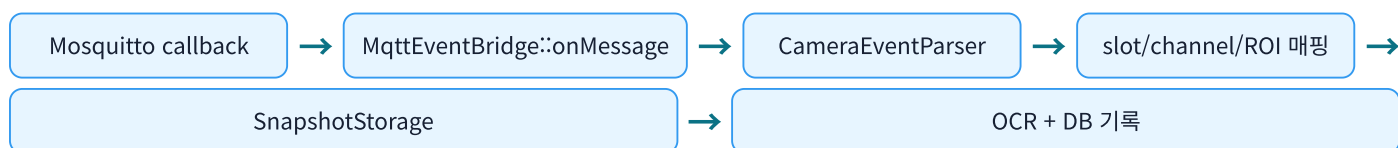
계층	책임	대표 코드
입력	RTSP 프레임, 카메라 MQTT, BestShot metadata, 홀센서 MQTT test 메시지	camera , mqtt , bestshot , sensor
처리	이벤트 해석, ROI 저장, 이미지 전처리, OCR, 주차 상태 및 타이머	event , snapshot , ocr , parking , timer
저장	차량, 주차면, 세션, 이미지, 이벤트 메타데이터	database , db/schema.sql
출력	Qt MQTT 상태·위반 알림과 HTTP/HTTPS 상태·이미지 조회 API	EventManager , MqttEventBridge , ParkingHttpServer

2. 서버 시작과 종료 흐름

```
src/main.cpp
├─ AppConfig::loadFromEnv()
├─ CameraChannel 생성
├─ EventDatabase::open()
├─ ParkingHttpServer::start()           [설정 시]
├─ RtspStreamReceiver::start()
├─ waitForInitialFrames()
├─ OcrWorker::start()
├─ BestShotReceiver::start()
├─ MqttEventBridge::start()
├─ EventManager publisher를 Qt MQTT에 연결
├─ 활성 EV/PHEV 세션 타이머 복구
├─ SIGINT/SIGTERM까지 대기
└─ MQTT → BestShot → OCR → RTSP → HTTP → DB 순서로 종료
```

RTSP 최초 프레임을 확보한 뒤 MQTT를 받는다. 이벤트가 먼저 들어오면 저장할 영상이 없기 때문에 `main.cpp`의 초기화 순서가 중요하다.

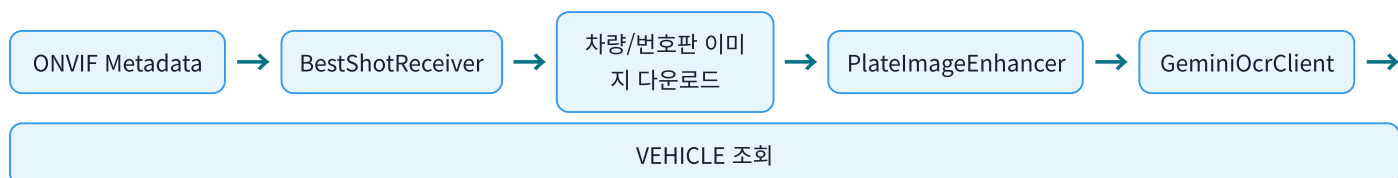
3. 카메라 MQTT·IVA Snapshot 흐름



카메라 MQTT topic/payload

- 이벤트 종류와 source ID 파싱
- IVA area 설정에서 slot_id와 channel_id 확인
- 해당 채널의 최신 full frame 복사
- ROI crop 원본 저장
- OpenCV enhanced 저장
- OcrWorker 큐 등록
- IMAGE_LOG / EVENT_LOG 기록

4. BestShot·번호판 OCR·EV 판별 흐름



```
Vehicle BestShot
→ ParkingTriggerCoordinator로 slot_id 결정
→ PARKING_SESSION ACTIVE 생성
→ IMAGE_LOG에 차량 이미지 연결
```

```
Plate BestShot
→ 같은 object/session과 연결
→ Resize / CLAHE / Denoise / Sharpen 전처리
→ Gemini HTTPS OCR
→ normalizePlateNumber()
→ VEHICLE.is_ev / is_phev 조회
→ OCR 결과와 판별 이벤트 저장
```

Gemini는 번호판 문자열만 추출한다. EV/PHEV 여부는 반드시 로컬 `VEHICLE` 테이블에서 판별한다. OCR 실패를 `NON_EV`로 간주하지 않는다.

5. 센서·주차 상태 흐름

```
STM32/UART 메시지 (향후 실제 장치 연결)
현재 parking/sensor/hall MQTT 또는 실제 UART/LoRa 입력
→ MqttEventBridge 또는 SensorLinkManager
→ HallParkingService::handleLine()
→ SensorProtocolParser::parse()
→ SensorProtocolMessage
→ ParkingSensorEventAdapter::adapt()
→ SensorSlotIndex에서 sensor_id → slot_id 매핑
→ ParkingSensorSequenceGuard 중복/역순 검사
→ parking::ParkingSlotManager::apply()
→ OCCUPIED: 최신 RTSP ROI 저장 → ACTIVE 세션 → OCR
→ VACANT: 세션 종료 → 위반 전이면 이미지/IMAGE_LOG 삭제
```

MQTT test, UART line, LoRa CRC frame은 전송 계층만 다르다. 세 입력 모두 검증된 센서 문자열을 같은 `HallParkingService::handleLine()`에 전달한다.

`parking/`은 센서 기반 상태 전이 도메인이고, `parking_timer/`는 EV/PHEV 장기 점유 시간 관리다. 이름이 비슷하지만 책임이 다르다.

6. 장기 점유 타이머 흐름

```
OCR callback → ParkingSlotManager::handleRecognizedSession(session_id, "EV01", plate)
→ EventDatabase::classifyVehicle()
→ EventDatabase::findActiveBySlot("EV01")
→ 기존 ACTIVE session_id를 그대로 사용
→ TimerManager::schedule()
→ deadline 도달
→ EvidenceProvider가 최신 RTSP ROI 저장
→ EventDatabase::markViolation()
→ VIOLATION_TRIGGERED 이벤트 → Qt MQTT

handleExit("EV01")
→ EventDatabase::departActiveBySlot()
→ 상태 ENDED, exit_time/duration_sec 저장
→ 큐의 과거 노드는 DB 조건부 UPDATE로 무효화
```

타이머와 DB는 `int zone_id` 가 아니라 `std::string slot_id` 를 사용한다. 따라서 EV01 이 숫자 변환 없이 그대로 저장·조회된다.

7. DB와 Qt HTTP/HTTPS 흐름

테이블	역할
VEHICLE	번호판, EV 여부, PHEV 여부를 보관하는 차량 마스터
PARKING_SLOT	EV01 같은 주차면과 점유 상태
PARKING_SESSION	입차, 위반, 출차, 점유시간 기록
IMAGE_LOG	원본·전처리·BestShot·타이머 이미지 경로와 OCR 결과
EVENT_LOG	카메라·OCR·주차 상태 이벤트와 처리 여부

```
Qt GET /api/v1/parking-slots
→ ParkingHttpServer route
→ EventDatabase::listParkingSlots()
→ db_visit_parking_slots()
→ JSON 응답
```

```
Qt GET /api/v1/images/{id}/enhanced
→ IMAGE_LOG 경로 조회
→ data root 내부 경로인지 검증
→ 이미지 파일 응답
```

```
Pi MQTT → parking/v1/events/{slot_id} (QoS 1, retain false)
→ parking/v1/state/{slot_id} (QoS 1, retain true)
```

```
Qt → payload의 session_images_url을 HTTP GET
→ original_url / enhanced_url을 다시 GET
```

이미지 binary는 MQTT로 전송하지 않는다. MQTT는 상태와 URL을 전달하고 실제 파일은 HTTP로 pull한다.

8. 최상위 폴더별 역할

폴더	역할	학습 포인트
<code>include/</code>	공개 타입, 클래스 인터페이스, 함수 선언	구현 전에 책임과 의존관계를 파악할 때 먼저 읽는다.
<code>src/</code>	서버와 타이머의 C/C++ 구현	<code>main.cpp</code> 에서 시작해 호출되는 모듈을 따라간다.
<code>tests/</code>	단위·통합·타이머 테스트 및 fixture	클래스의 기대 동작을 가장 빠르게 확인한다.
<code>db/</code>	단일 SQLite 스키마와 seed 데이터	테이블 관계와 실제 EV01 ID를 확인한다.
<code>config/</code>	타이머 및 센서-슬롯 실행 설정	DB-제한시간과 HALL01~04 → EV01~04 매핑을 확인한다.
<code>driver/</code>	프로젝트 전용 Linux 커널 모듈	<code>parking_alert.ko</code> 가 <code>/dev/parking_alert</code> 를 생성한다.
<code>tools/</code>	DB 백업·초기화·조회, 문서 생성 자동화	운영 보조 도구이며 서버 런타임 코드는 아니다.
<code>docs/</code>	아키텍처, DB, MQTT, HTTP, 테스트 및 자동 API 문서	설계 의도와 현재/목표 구현 차이를 확인한다.
<code>data/</code>	실행 중 생성되는 DB-Snapshot·BestShot·전처리 파일	소스가 아니라 런타임 데이터이며 환경에 따라 없을 수 있다.
<code>CMakeLists.txt</code>	서버·테스트·타이머·문서 타깃 정의	어떤 소스가 어떤 실행 파일에 링크되는지 확인한다.

9. 모듈 및 파일별 역할

app — 실행 설정

파일	역할	주요 함수/타입
<code>include/app/AppConfig.hpp</code>	환경변수에서 읽을 모든 서버 설정 구조	<code>AppConfig</code> , <code>RtspChannelConfig</code> , <code>IvaAreaConfig</code>
<code>src/app/AppConfig.cpp</code>	RTSP, MQTT, DB, OCR, HTTP, IVA ROI 환경 변수 파싱과 기본값	<code>AppConfig::loadFromEnv()</code>

camera — 연속 RTSP 프레임

파일	역할	주요 함수/타입
<code>include/camera/CameraChannel.hpp</code>	채널 ID, RTSP URL, 최신 프레임과 mutex를 공유하는 상태 객체	<code>CameraChannel</code>
<code>include/camera/RtspStreamReceiver.hpp</code>	채널별 RTSP 수신 스레드 인터페이스	<code>start()</code> , <code>stop()</code> , <code>waitForInitialFrames()</code>
<code>src/camera/RtspStreamReceiver.cpp</code>	OpenCV VideoCapture 연결·재접속·최신 프레임 갱신	<code>captureLoop()</code>

mqtt + event — 카메라 이벤트

파일	역할	주요 함수/타입
----	----	----------

<code>include/mqtt/MqttEventBridge.hpp</code>	Mosquitto 연결, 카메라/홀 구독, Qt용 이벤트 발행 인터페이스	<code>start()</code> , <code>stop()</code> , <code>onMessage()</code> , <code>publishQtEvent()</code>
<code>src/mqtt/MqttEventBridge.cpp</code>	카메라 이벤트는 IVA 흐름, 홀 topic 은 HallParkingService로 라우팅	연결/메시지 callback, <code>onMessage()</code> , <code>publishQtEvent()</code>
<code>include/event/CameraEvent.hpp</code>	정규화된 카메라 이벤트 값 객체	<code>CameraEvent</code>
<code>include/event/CameraEventParser.hpp</code>	topic/payload 파서 인터페이스	<code>parse()</code> , <code>parseSourceId()</code> , <code>parseEventType()</code>
<code>src/event/CameraEventParser.cpp</code>	Hanwha/ONVIF topic에서 채널·source·이벤트 종류 추출	문자열 파싱 구현
<code>include/event/EventPayloadBuilder.hpp</code>	정규 이벤트 JSON 생성 인터페이스	<code>build()</code>
<code>src/event/EventPayloadBuilder.cpp</code>	Qt/MQTT 전달용 JSON 직렬화	<code>EventPayloadBuilder::build()</code>

snapshot — IVA ROI 이미지 저장

파일	역할	주요 함수/타입
<code>include/snapshot/SnapshotStorage.hpp</code>	최신 프레임 대기, ROI 정규화, 원본·전처리 저장 인터페이스	<code>saveIvaAreaSnapshot()</code> , <code>waitForFullFrame()</code>
<code>src/snapshot/SnapshotStorage.cpp</code>	<code>data/snapshots/ch*/EV*/scene enhanced</code> 경로 생성과 OpenCV 저장	ROI crop 및 파일명 생성

bestshot — 차량·번호판 BestShot

파일	역할	주요 함수/타입
<code>include/bestshot/BestShotReceiver.hpp</code>	채널별 metadata 수신, object/session 상관관계 인터페이스	<code>start()</code> , <code>stop()</code> , <code>receiveLoop()</code>
<code>src/bestshot/BestShotReceiver.cpp</code>	metadata XML 파싱, 이미지 URL 다운로드, vehicle/plate 연결, OCR 요청	BestShot 분기와 pending plate 처리

ocr — 전처리와 Gemini 번호판 인식

파일	역할	주요 함수/타입
<code>include/ocr/GeminiOcrClient.hpp</code>	Gemini HTTPS OCR 요청/응답 타입	<code>recognizePlate()</code> , <code>configured()</code>
<code>src/ocr/GeminiOcrClient.cpp</code>	libcurl HTTPS 호출, timeout, JSON 응답 파싱과 fallback 모델	요청 body와 OCR 결과 파싱
<code>include/ocr/OcrWorker.hpp</code>	OCR 비동기 작업 큐, 세션별 취소와 worker 인터페이스	<code>start()</code> , <code>stop()</code> , <code>enqueue()</code> , <code>cancelSession()</code>
<code>src/ocr/OcrWorker.cpp</code>	전처리→Gemini→정규화→DB 반영 파이프라인	<code>run()</code>
<code>include/ocr/PlateImageEnhancer.hpp</code>	번호판/IVA 이미지 전처리 결과와 공개 함수	<code>preprocessPlateImage()</code> , <code>enhancePlateImage()</code> , <code>enhanceIvaSceneImage()</code>
<code>src/ocr/PlateImageEnhancer.cpp</code>	Resize, CLAHE, denoise, sharpen 등 OpenCV C++ 전처	전처리 단계별 구현

	리	
<code>include/ocr/PlateNormalizer.hpp</code>	OCR 문자열 정규화·형식 검사 인터페이스	<code>normalizePlateNumber()</code> , <code>isPlausibleKoreanPlate()</code>
<code>src/ocr/PlateNormalizer.cpp</code>	공백·기호 제거와 한국 번호판 후보 검증	정규식/문자열 처리

database — SQLite 저장 계층

파일	역할	주요 함수/타입
<code>include/database/EventDatabase.hpp</code>	카메라·OCR·HTTP·타이머가 공유하는 단일 C++ DB 인터페이스	<code>EventDatabase</code> , <code>View/Record</code> 구조체
<code>src/database/EventDatabase.cpp</code>	카메라 이벤트, 센서 Snapshot, BestShot, OCR, HTTP 조회용 DB orchestration	<code>createEntryWithSnapshot()</code> , <code>createEntryWithBestShot()</code> , <code>deleteSessionImageRecords()</code>
<code>src/database/EventDatabaseTimer.cpp</code>	EV/PHEV 분류와 장기 점유 세션·위반·출차 트랜잭션	<code>insertParked()</code> , <code>markViolation()</code> , <code>departActiveBySlot()</code>
<code>include/database/db_manager.h</code>	SQLite C API 공개 함수와 조회 row 구조체	<code>db_open()</code> , CRUD/visitor 함수
<code>src/database/db_manager.c</code>	prepared statement 기반 실제 SQLite CRUD	VEHICLE/SESSION/IMAGE/EVENT 쿼리
<code>db/schema.sql</code>	프로젝트 단일 DB 스키마	5개 테이블, FK, 인덱스, 활성 세션 UNIQUE
<code>db/seed.sql</code>	EV/PHEV/일반차 및 EV01~EV04/P01~P04 초기 데이터	<code>INSERT OR IGNORE</code>

http — Qt 조회 API

파일	역할	주요 함수/타입
<code>include/http/ParkingHttpServer.hpp</code>	HTTP/HTTPS 서버 설정과 생명주기	<code>start()</code> , <code>stop()</code> , <code>usesTls()</code>
<code>src/http/ParkingHttpServer.cpp</code>	health, 슬롯, 세션 이미지, 원본/전처리 파일 route 구현	<code>registerRoutes()</code>

parking — 센서 기반 주차 도메인

파일	역할	주요 함수/타입
<code>ParkingSlot.hpp</code>	슬롯 ID·종류·상태 값 객체	<code>ParkingSlot</code>
<code>ParkingSensorEvent.hpp</code>	슬롯에 매핑된 OCCUPIED/VACANT 이벤트	<code>ParkingSensorEvent</code>
<code>ParkingSlotConfig.hpp/.cpp</code>	JSON 슬롯·센서·카메라·ROI 설정 로더	<code>ParkingSlotConfigLoader::load()</code>
<code>SensorSlotIndex.hpp/.cpp</code>	<code>sensor_id</code> 에서 <code>slot_id</code> 와 설정을 찾는 인덱스	<code>find()</code>
<code>ParkingSensorSequenceGuard.hpp/.cpp</code>	센서 sequence 중복·역순 메시지 차단	<code>accept()</code> , <code>reset()</code>

<code>ParkingOccupancySession.hpp/.cpp</code>	한 주차 세션의 Active/Completed 상태와 시간	<code>complete()</code> , <code>duration()</code>
<code>ParkingSlotManager.hpp/.cpp</code>	센서 이벤트로 주차면 상태 전이 결정	<code>apply()</code> , <code>findSlot()</code>
<code>ActiveParkingSessionIndex.hpp/.cpp</code>	활성 세션을 slot_id 기준으로 관리	<code>apply()</code> , <code>find()</code> , <code>clear()</code>
<code>ParkingTriggerCoordinator.hpp/.cpp</code>	IVA와 홀센서/BestShot 시간 상관관계 및 중복 억제	<code>recordCameraIva()</code> , <code>recordHallState()</code> , <code>claimSlotForVehicle()</code>

sensor — STM32 메시지 경계

파일	역할	주요 함수/타입
<code>SensorProtocolMessage.hpp</code>	UART 한 줄 메시지의 파싱 결과	<code>SensorProtocolMessage</code>
<code>SensorProtocolParser.hpp/.cpp</code>	partial/multi-line byte stream buffering과 필드 검증	<code>push()</code> , <code>parseLine()</code>
<code>ParkingSensorEventAdapter.hpp/.cpp</code>	프로토콜 메시지를 주차 도메인 이벤트로 변환	<code>adapt()</code>
<code>HallParkingService.hpp/.cpp</code>	홀 입력을 실제 Snapshot·세션·OCR·타이머 및 조기 정리 흐름으로 연결	<code>handleLine()</code> , <code>handleOccupied()</code> , <code>handleVacant()</code>

device — UART·LoRa·Character Device 경계

파일	역할	주요 함수/타입
<code>UartDriver.hpp/.cpp</code>	POSIX termios raw 8N1, poll, partial read/write로 <code>/dev/tty*</code> 제어	<code>connect()</code> , <code>readSome()</code> , <code>writeAll()</code>
<code>LoRaDriver.hpp/.cpp</code>	투명 UART LoRa byte stream의 SOF·길이·sequence·CRC16 frame 처리	<code>encode()</code> , <code>consume()</code> , <code>send()</code>
<code>SensorLinkManager.hpp/.cpp</code>	UART 재연결 thread, line/frame 수신 및 Hall handler 전달	<code>start()</code> , <code>stop()</code> , <code>sendAlertCommand()</code>
<code>LinuxDriverAdapter.hpp/.cpp</code>	C++ 서버에서 <code>/dev/parking_alert</code> open/ioctl/close 캡슐화	<code>setSlot()</code> , <code>clearSlot()</code> , <code>state()</code>

driver — /dev/parking_alert 커널 모듈

파일	역할	주요 인터페이스
<code>driver/parking_alert/parking_alert.c</code>	32개 슬롯 bit 상태, 중복 억제, blocking read와 poll 통지	<code>open/read/write/poll/ioctl</code>
<code>include/uapi/parking_alert.h</code>	커널과 사용자 공간이 공유하는 versioned ABI	<code>parking_alert_command</code> , <code>parking_alert_state</code>
<code>driver/parking_alert/Kbuild</code>	현재 Raspberry Pi 커널 헤더에 맞는 module 빌드	<code>parking_alert.ko</code>

parking_timer — EV/PHEV 장기 점유

파일	역할	주요 함수/타입
----	----	----------

EventDatabase.hpp	통합 database::EventDatabase 호환 별칭	using EventDatabase
Types.hpp/.cpp	차량 분류, 타이머 로그·입차 결과, UTC 시각	VehicleCategory , LogRecord , utcNow()
RuntimeConfig.hpp/.cpp	타이머 DB 경로와 timeout 설정	load() , applyEnvironment()
EventManager.hpp/.cpp	타이머 결과를 JSON 로그와 외부 Qt MQTT publisher로 전달	setPublisher() , publish()
TimerManager.hpp/.cpp	우선순위 큐, 재시작 복구, 만료 증거 provider, DB 오류 재시도	schedule() , restoreActiveSessions() , processExpired()
ParkingSlotManager.hpp/.cpp	EV/PHEV 입차·출차와 타이머 orchestration	handleEntry() , handleExit()
src/timer/main.cpp	별도 타이머 CLI, demo, interactive 명령	main() , runDemo() , runInteractive()

util — 공통 보조 함수

파일	역할	주요 함수
Logger.hpp/.cpp	레벨이 포함된 headless 로그 출력	logInfo() , logWarn() , logError()
StringUtil.hpp/.cpp	JSON 문자열 escape	jsonEscape()
TimeUtil.hpp/.cpp	로그·파일명용 현재 시각	nowIsoString() , nowStringForFilename()
UrlMasker.hpp/.cpp	로그에 RTSP/HTTP 인증정보가 노출되지 않도록 마스크	hideUrlForLog()

테스트·설정·도구

파일	검증/역할
tests/unit/ParkingDomainTest.cpp	슬롯 설정, 센서 매핑, 상태 전이, 세션 인덱스 단위 테스트
tests/unit/SensorParkingPipelineTest.cpp	partial/multi-line UART 파싱부터 주차 상태까지 파이프라인 테스트
tests/timer/timer_tests.cpp	EV01 입차, 중복, EV/PHEV 분류, 위반, 출차, worker 예외 테스트
tests/integration/DbManagerTest.c	SQLite C CRUD 통합 확인
tests/integration/HttpApiTest.cpp	슬롯/이미지 HTTP API 응답과 경로 보안 확인
tests/integration/GeminiOcrTest.cpp	실제 환경변수 기반 Gemini OCR 수동 통합 검증
tests/integration/HallTimerIntegrationTest.cpp	가짜 frame과 SQLite로 입차·중복·조기삭제·위반 Snapshot·증거보존 통합 검증
tests/unit/UartLoRaDriverTest.cpp	PTY 기반 UART 양방향, partial line, LoRa CRC·재동기화 및 alert 송신 검증
tests/fixtures/parking_slots.json	EV01~EV04 슬롯·센서·ROI 테스트 설정
config/timer.conf	타이머 DB 경로와 제한시간 예시
config/parking_slots.json	HALL01~04 와 EV01~04 센서·슬롯 매핑
tools/reset_db.sh	db/schema.sql 과 db/seed.sql 로 DB 재생성
tools/backup_db.sh	SQLite DB 안전 백업
tools/inspect_db.sh	주요 테이블 내용을 CLI로 조회

<code>tools/build_docs.sh</code>	코드 가이드 HTML을 PDF로 자동 변환
<code>tools/parking_link_tool.cpp</code>	LoRa frame encode/send/listen용 C++ 실기기 진단 도구
<code>tools/parking_alert_ctl.c</code>	Character device status/set/clear/watch 및 ioctl/write/poll 검증 도구
<code>docs/CAMERA_MQTT_CAPTURE_PROTOCOL.md</code>	카메라 요청형 촬영 목표 규약, ROI, 응답·재시도·조기삭제 명세
<code>docs/GEMINI_OCR_GUIDE.md</code>	OpenCV 전처리부터 Gemini HTTPS·정규화·DB·수동시험까지의 운영 가이드
<code>docs/PARKING_ALERT_DRIVER.md</code>	<code>/dev/parking_alert</code> ABI·빌드·권한·실기기 테스트 가이드

10. 핵심 함수 참조

함수	입력 → 처리 → 출력
AppConfig::loadFromEnv()	환경변수 → URL/timeout/ROI/default 검증 → 전체 런타임 설정
RtspStreamReceiver::captureLoop()	채널 URL → RTSP 연결·재시도·frame clone → CameraChannel 최신 프레임
MqttEventBridge::onMessage()	topic/payload → 카메라 이벤트 또는 홀 test 입력 분기 → 각 업무 흐름
MqttEventBridge::publishQtEvent()	topic/JSON/QoS/retain → Mosquitto publish → Qt 상태·이벤트
HallParkingService::handleLine()	홀 한 줄 → parse·mapping·중복검사 → 입차 Snapshot/OCR 또는 출차 정리
UartDriver::readSome()	/dev/tty* → poll·partial read → 수신 byte 0..N개
LoRaDriver::consume()	임의 byte chunk → SOF·길이·CRC 검증과 재동기화 → frame 0..N개
SensorLinkManager::run()	UART 재연결 → line/LoRa 분기 → 센서 handler callback
parking_alert_apply_command()	slot 명령 검증 → 32-bit 상태 변경 → generation 증가와 wait queue 통지
LinuxDriverAdapter::setSlot()	C++ 주차면 bit → versioned ioctl → /dev/parking_alert 상태 갱신
SnapshotStorage::saveIvaAreaSnapshot()	채널+slot+ROI → 최신 frame crop·전처리 → scene/enhanced 경로
BestShotReceiver::receiveLoop()	metadata stream → object 종류/URL/ID 파싱 → 이미지 저장·세션 연결·OCR
OcrWorker::enqueue()	session/slot/image → thread-safe queue → worker 깨움
OcrWorker::run()	이미지 작업 → 전처리·Gemini·정규화 → DB OCR/EV 판별 이벤트
OcrWorker::cancelSession()	조기 출차 session_id → 대기/처리 작업 취소 표시 → 뒤늦은 DB·타이머 반영 차단
GeminiOcrClient::recognizePlate()	이미지 → base64 HTTPS 요청 → OCR 문자열/상태
normalizePlateNumber()	Gemini 응답 → 불필요 문자 제거 → 정규화 번호판
EventDatabase::createEntryWithBestShot()	slot/vehicle image → OCCUPIED+ACTIVE session+IMAGE_LOG → session_id
EventDatabase::createEntryWithSnapshot()	홀 입차 Snapshot → ACTIVE session+IMAGE_LOG+EVENT_LOG → 같은 session_id
EventDatabase::applyPlateOcr()	plate/confidence → VEHICLE 조회·세션 연결 → EV/PHEV/NON_EV/UNKNOWN
EventDatabase::insertParked()	plate/EV01/time/image → transaction → ACTIVE session 및 TIMER_ENTRY 이미지
EventDatabase::markViolation()	session/deadline/image → 조건부 transaction → VIOLATION 및 증거 이미지
EventDatabase::departActiveBySlot()	EV01/출차시각 → 활성 세션 검색·duration 계산 → ENDED LogRecord

ParkingHttpServer::registerRoutes()	DB/파일 저장소 → httplib route 등록 → Qt JSON/이미지 응답
SensorProtocolParser::push()	임의 길이 byte chunk → newline buffering·필드 검증 → 메시지 0..N개
parking::ParkingSlotManager::apply()	센서 이벤트 → 상태 전이·중복 판단 → ParkingTransitionResult
ParkingTriggerCoordinator::claimSlotForVehicle()	채널/최근 IVA·홀 이벤트 → 상관시간 검사 → BestShot 대상 slot_id
parking_timer::ParkingSlotManager::handleEntry()	EV01/번호판 → 분류·중복 검사·세션 생성 → 타이머 예약
TimerManager::schedule()	session/slot/delay → 최소 우선순위 큐 → worker 알림
TimerManager::processExpired()	만료 노드 → 최신 RTSP 증거 요청 → 조건부 DB 위반 처리 → Qt publisher 또는 재시도

11. 테스트와 디버깅

빌드와 전체 테스트

```
cmake -S . -B build -DCMAKE_BUILD_TYPE=Debug
cmake --build build -j2
ctest --test-dir build --output-on-failure
```

문서 재생성

```
cmake --build build --target docs

# 결과
Pi_Server_Code_Guide.pdf
```

문제별 확인 위치

증상	우선 확인
RTSP 프레임 없음	환경변수 URL, RtspStreamReceiver 재접속 로그, URL 마스킹 결과
MQTT 이벤트 없음	Mosquitto broker, subscribe topic, 카메라 publication, MqttEventBridge 로그
IVA 사진 없음	IvaAreaConfig 의 channel/slot/ROI, 최신 frame 존재 여부, 저장 권한
BestShot 없음	metadata profile, image URL, object type, BestShotReceiver download 로그
OCR 실패	Gemini key/model/timeout, enhanced 이미지, 정규화 전 raw 결과
EV 판별 오류	VEHICLE 의 plate_number/is_ev/is_phev, 정규화된 번호판
EV01 세션 조회 실패	모든 계층이 문자열 slot_id 를 사용하는지, seed와 fixture ID 일치 여부
Qt 이미지 오류	IMAGE_LOG 경로, HTTP data root, 파일 존재 여부와 크기 제한

12. 학습 권장 순서

- src/main.cpp 에서 객체 생성과 시작/종료 순서를 읽는다.
- include/ 헤더로 각 클래스의 공개 책임만 먼저 익힌다.

3. MQTT 이벤트 한 건을 기준으로 `MqttEventBridge` → `SnapshotStorage` → `OcrWorker` → `EventDatabase` 를 추적한다.
4. BestShot 한 건을 기준으로 `BestShotReceiver` → `OcrWorker` → DB 를 추적한다.
5. `tests/` 에서 기대 상태 전이를 확인한 뒤 구현 `cpp`를 읽는다.
6. 헤더의 Doxygen 주석과 구현 파일에서 `private helper`와 실제 호출 위치를 확인한다.

문서와 코드가 다르면 코드와 테스트가 기준이다. 구조가 변경되면 이 HTML의 설계 설명을 갱신하고 `cmake --build build --target docs` 로 PDF와 API 문서를 다시 생성한다.